

Scaling vs. Normalization: What's the difference?

in scaling, you're changing the range of your data, 😊

in normalization, you're changing the shape of the distribution of your data. 😞

Scaling

Scaling involves changing the range of feature values to a standard range, typically $[0, 1]$. It is commonly used when the features have different units or widely different ranges. Common scaling methods include:

Min-Max Scaling:

Rescales the feature to a fixed range, typically $[0, 1]$.

Standardization (Z-score normalization):

Centers the feature values around the mean(0) with a unit standard deviation.(1)

```
In [3]: import seaborn as sns
data=sns.load_dataset("tips")
data
```

```
Out[3]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

244 rows × 7 columns

```
In [4]: data.skew()
```

C:\Users\HP\AppData\Local\Temp\ipykernel_15540\1188251951.py:1: FutureWarning: Dropping of nuisance columns in DataFrame reductions (with 'numeric_only=None') is deprecated; in a future version this will raise TypeError. Select only valid columns before calling the reduction.

```
data.skew()
```

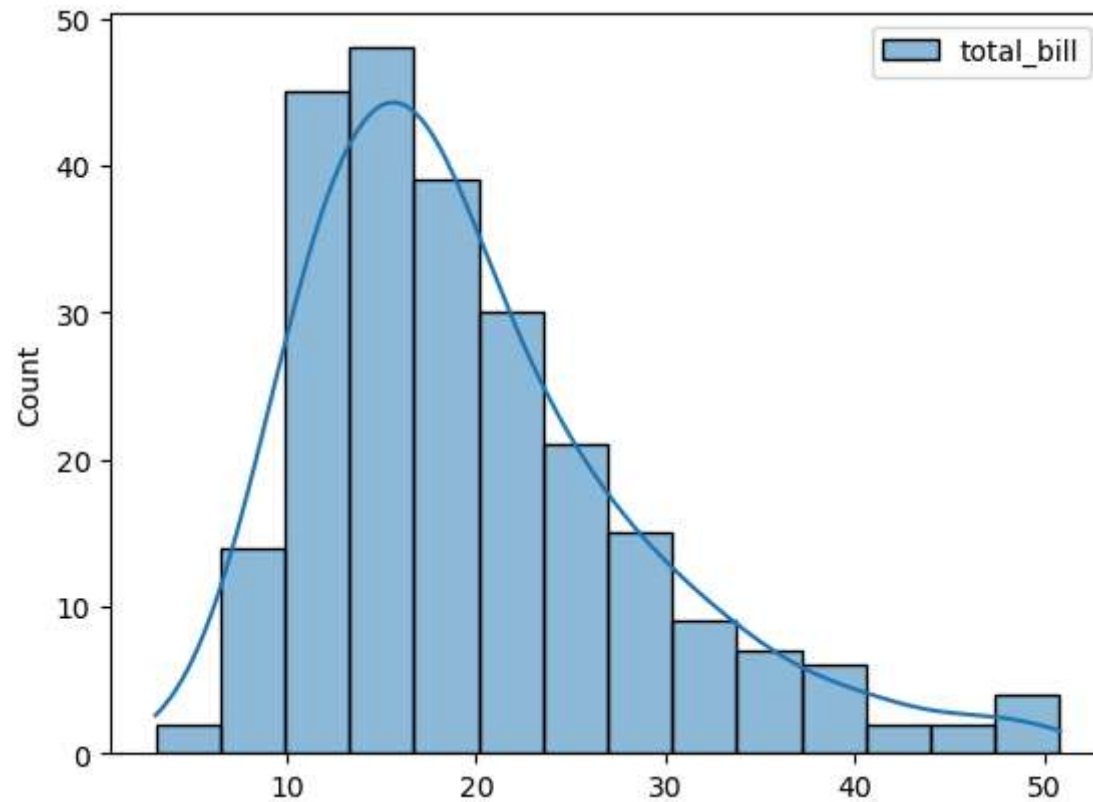
```
Out[4]: total_bill    1.133213
tip                1.465451
size              1.447882
dtype: float64
```

Min-Max Scaling:

```
In [5]: x=data[["total_bill"]]
```

```
In [6]: sns.histplot(x,kde=True)
```

```
Out[6]: <AxesSubplot:ylabel='Count'>
```



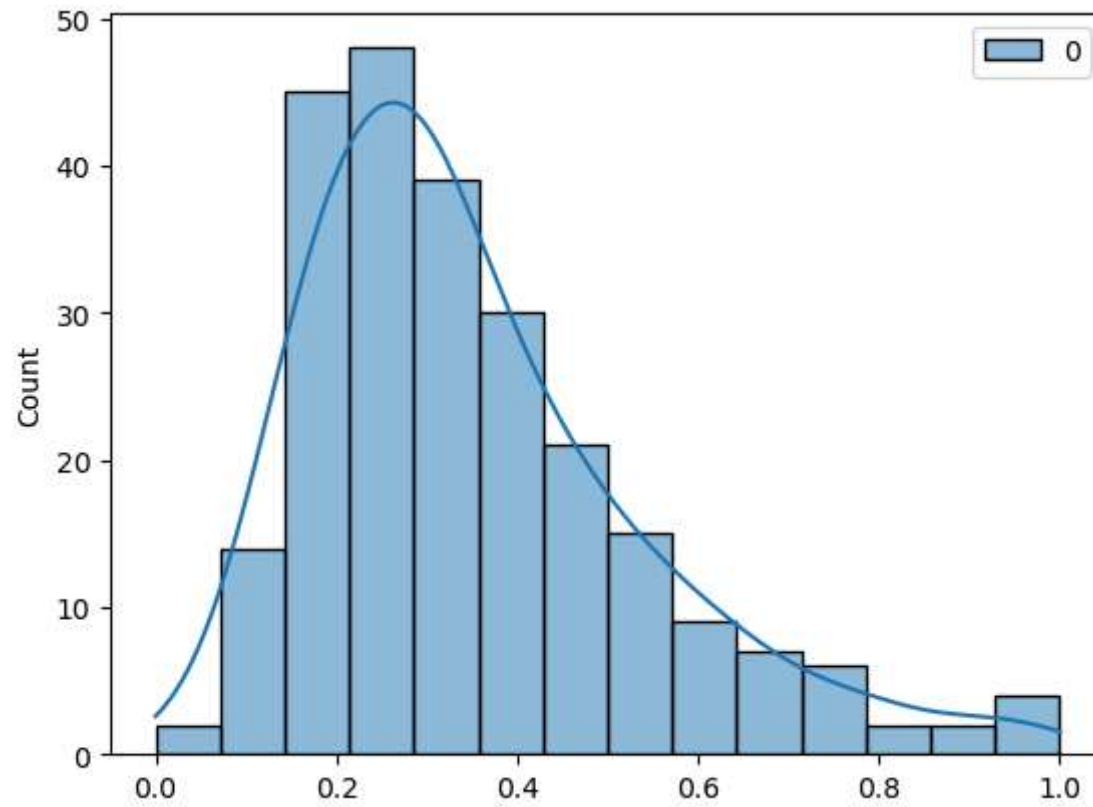
```
In [7]: import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler
```

```
# Min-Max Scaling
y = MinMaxScaler()
data_min_max_scaled = y.fit_transform(x)
data_min_max_scaled
```

```
Out[8]: array([[0.29157939],  
               [0.1522832 ],  
               [0.3757855 ],  
               [0.43171345],  
               [0.45077503],  
               [0.46543779],  
               [0.11939673],  
               [0.49874319],  
               [0.25073314],  
               [0.24528697],  
               [0.15081693],  
               [0.67427734],  
               [0.25869292],  
               [0.32174277],  
               [0.24633431],  
               [0.38772518],  
               [0.15207373],  
               [0.27691663],  
               [0.29116045],  
               [0.25001166]])
```

```
In [7]: sns.histplot(data_min_max_scaled,kde=True)
```

```
Out[7]: <AxesSubplot:ylabel='Count'>
```



Standardization (Z-score normalization):

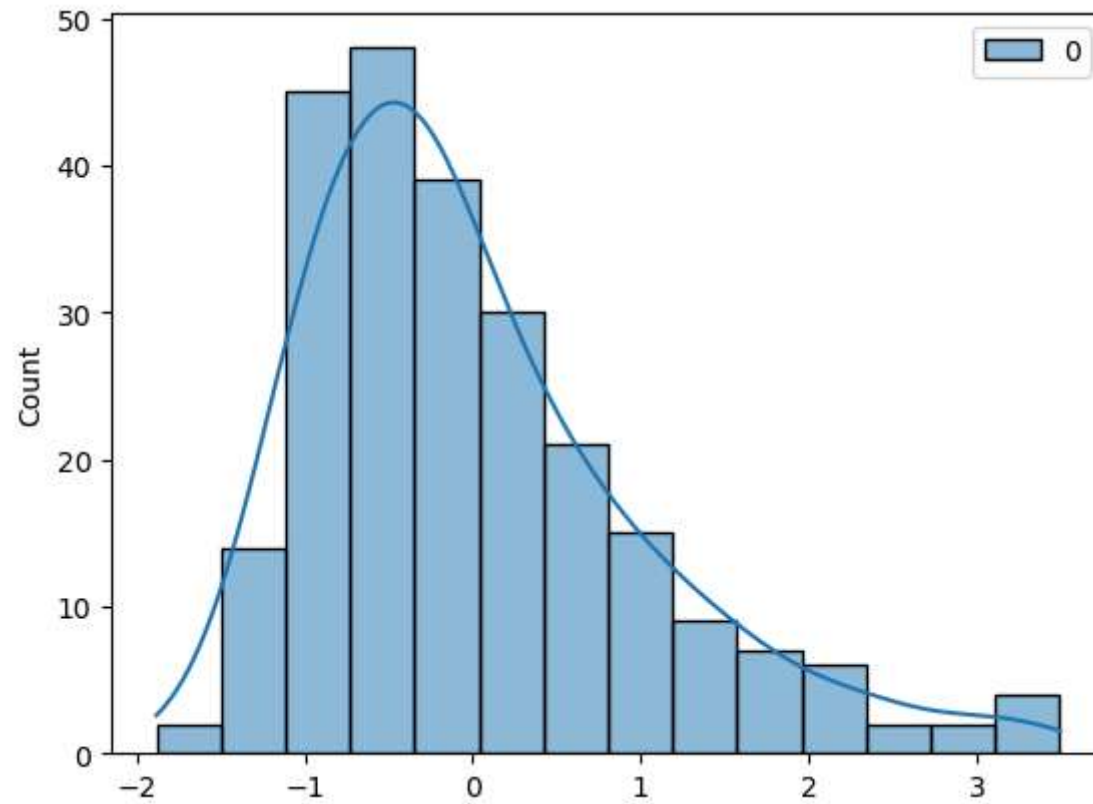
Centers the feature values around the mean with a unit standard deviation.

```
In [9]: # Standardization (Z-score normalization)
z = StandardScaler()
data_standard_scaled = z.fit_transform(x)
data_standard_scaled
```

```
[ 0.00000000e+00],
[-1.57988572e+00],
[-3.90126506e-01],
[ 3.33634302e-01],
[ 2.29442952e+00],
[ 8.43531108e-01],
[-8.73008911e-01],
[ 1.37779900e-01],
[-8.24608111e-01],
[-9.49549712e-01],
[-4.95932907e-01],
[ 2.75930233e+00],
[ 2.96489502e-01],
[ 1.27649500e-01],
[-4.98184107e-01],
[ 7.92486992e-02],
[ 6.10531905e-01],
[-1.74011304e-01],
[-6.16372108e-01],
[-6.51265709e-01],
[ 1.41101572e+00]
```

```
In [10]: sns.histplot(data_standard_scaled,kde=True)
```

```
Out[10]: <AxesSubplot:ylabel='Count'>
```



```
In [10]: #Notice that the shape of the data doesn't change,
```

Normalization

Scaling just changes the range of your data. Normalization is a more radical transformation. The point of normalization is to change your observations so that they can be described as a normal distribution.

Box-Cox.

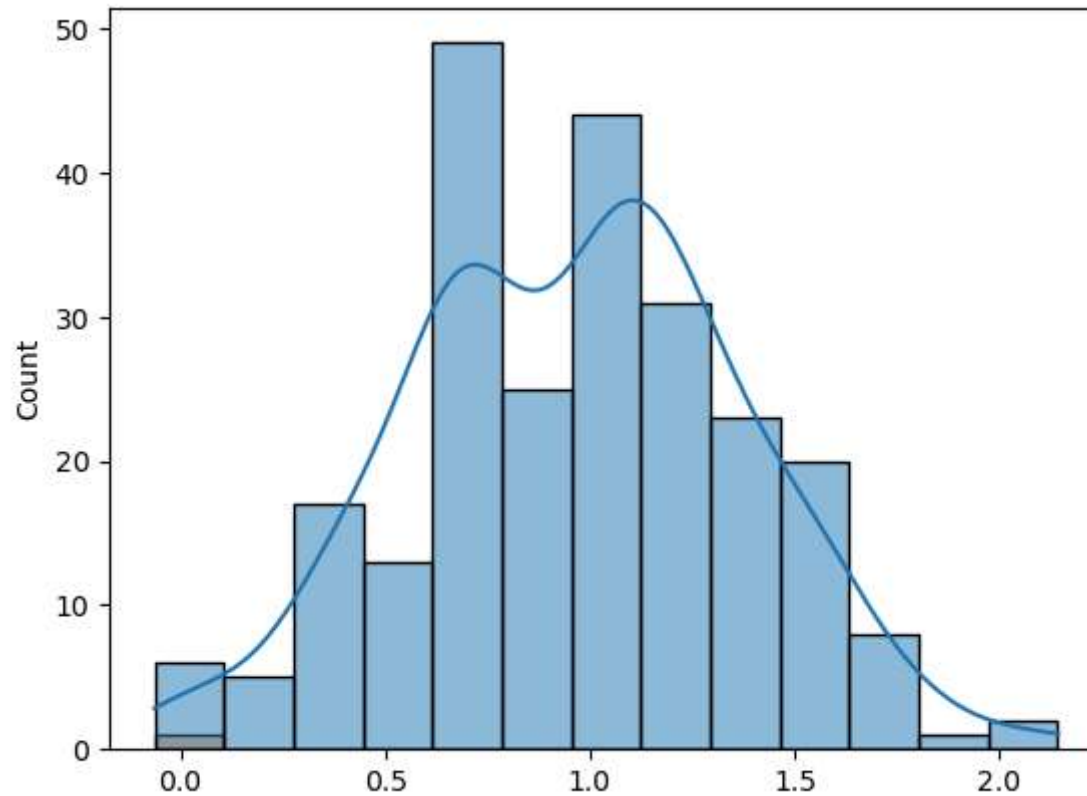
When your data is skewed (not normally distributed), applying a Box-Cox transformation can help make it more normally distributed.

```
In [11]: x=data["tip"]  
         from scipy import stats  
         normalized_data = stats.boxcox(x)
```



```
In [12]: sns.histplot(normalized_data,kde=True,legend=False)
```

```
Out[12]: <AxesSubplot:ylabel='Count'>
```



Correlation

Correlation measures the strength and direction of the relationship between two variables. It is a crucial concept in statistics and data analysis, used to identify and quantify the degree to which two variables are related.

Correlation Coefficient:

A numerical measure that indicates the extent of the relationship between two variables.

Values range from -1 to 1.

+1: Perfect positive correlation (variables move in the same direction).

-1: Perfect negative correlation (variables move in opposite directions).

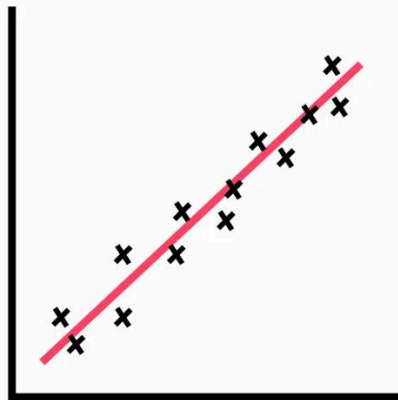
0: No correlation (no linear relationship). Correlation is used to measure the relationship between two variables.

Pearson Correlation

is best for linear relationships with continuous data.

Spearman Correlation

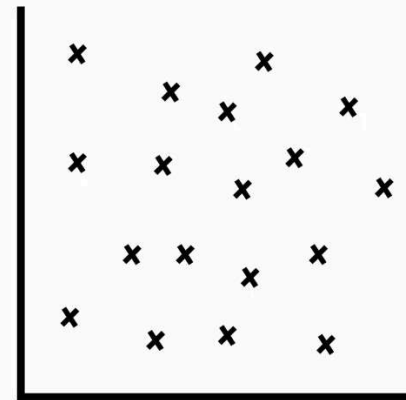
is best for monotonic relationships with ordinal data



Positive
Correlation



Negative
Correlation



No
Correlation

```
In [3]: import seaborn as sns
data=sns.load_dataset("tips")
data
```

Out[3]:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

244 rows × 7 columns

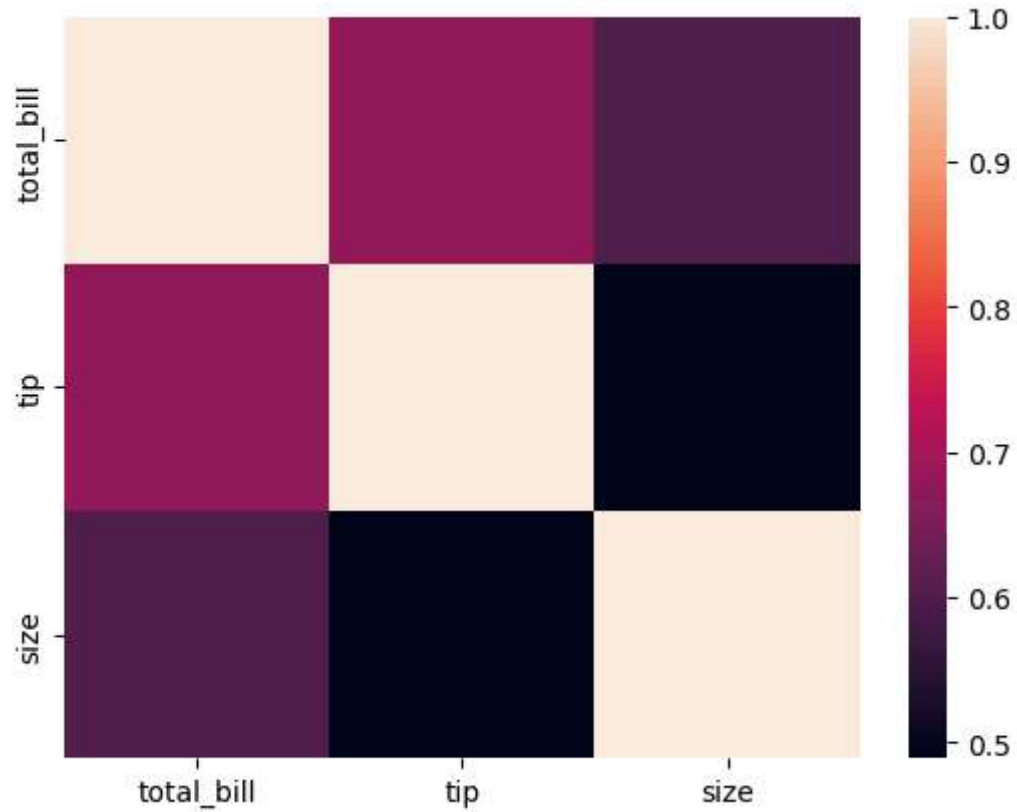
```
In [6]: nu=data.select_dtypes("number")
x = nu.corr()
x
```

Out[6]:

	total_bill	tip	size
total_bill	1.000000	0.675734	0.598315
tip	0.675734	1.000000	0.489299
size	0.598315	0.489299	1.000000

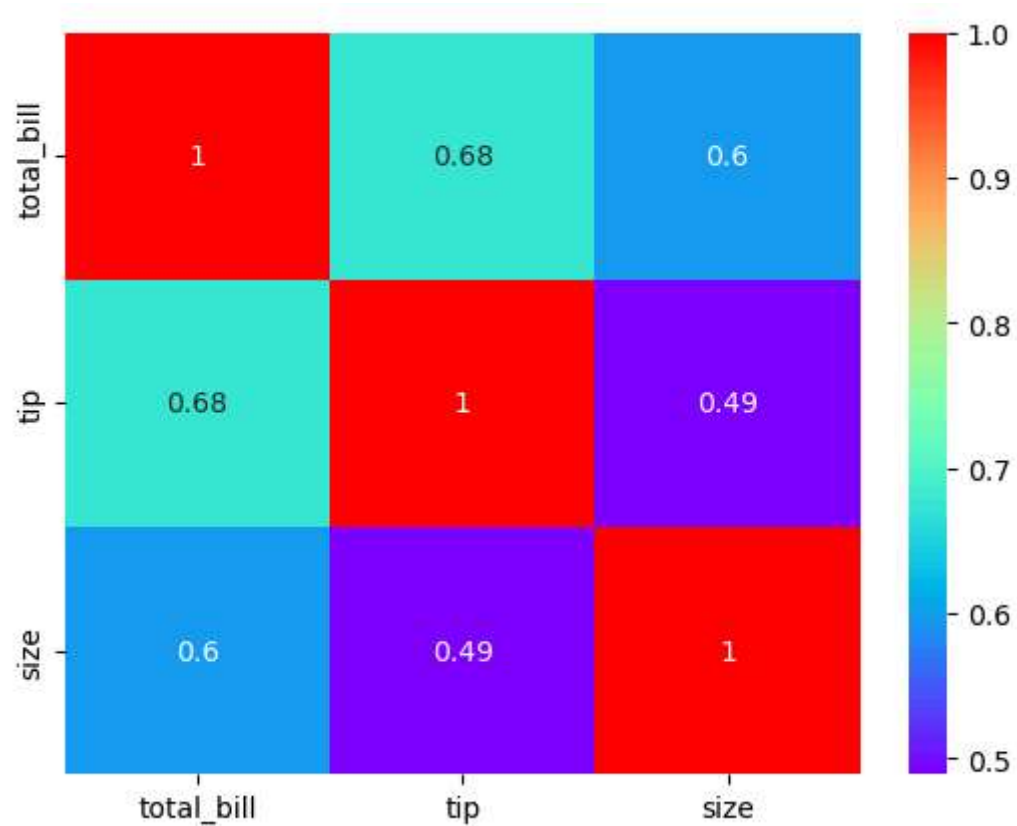
```
In [8]: import seaborn as sns
sns.heatmap(x) # this will give you a basic heat map
```

Out[8]: <AxesSubplot:>



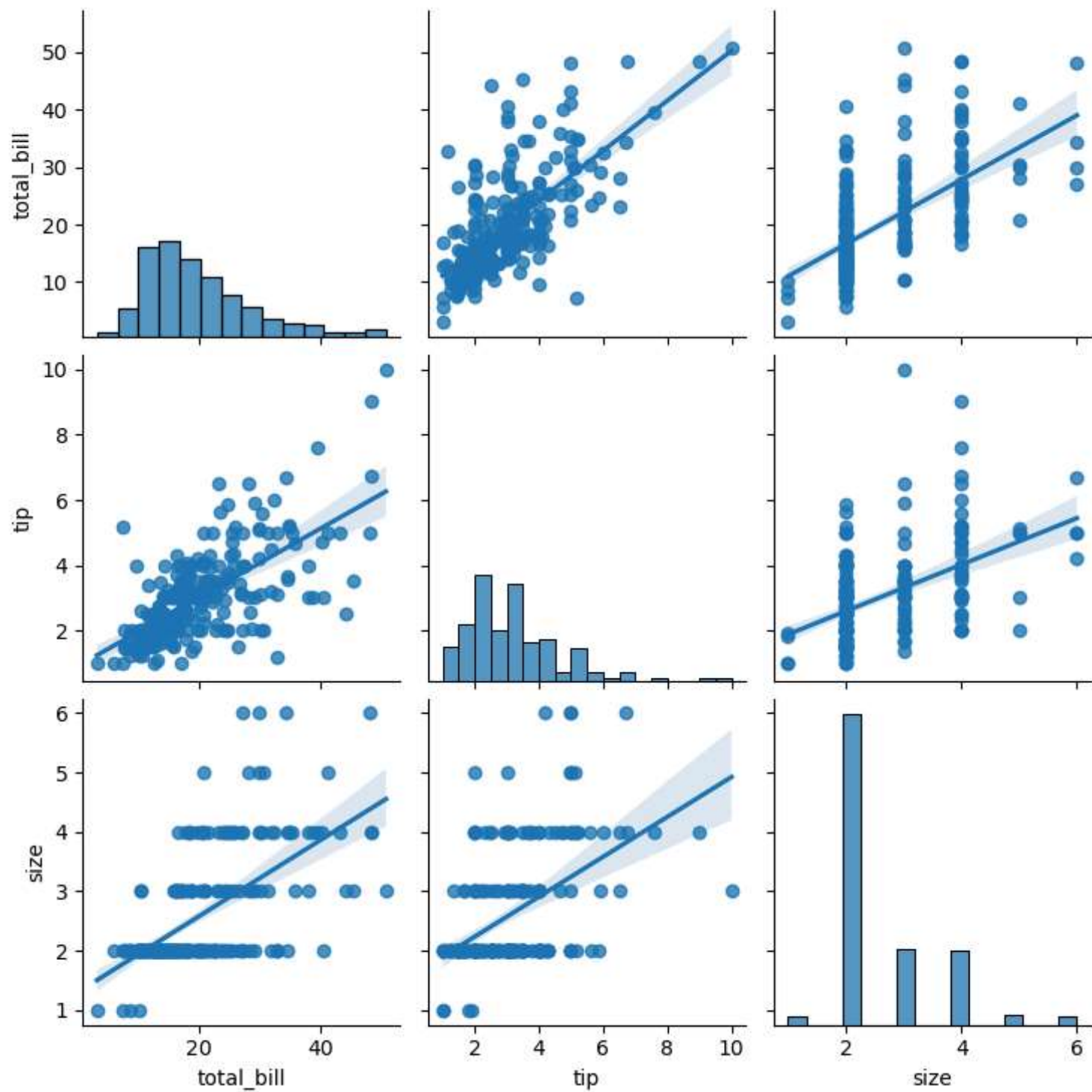
```
In [13]: import seaborn as sns
sns.heatmap(corr,      annot=True,cmap="rainbow") # this will give you a basic heat map
```

Out[13]: <AxesSubplot:>



```
In [15]: sns.pairplot(data, kind="reg",)
```

```
Out[15]: <seaborn.axisgrid.PairGrid at 0x1dcb9aee610>
```

In []:

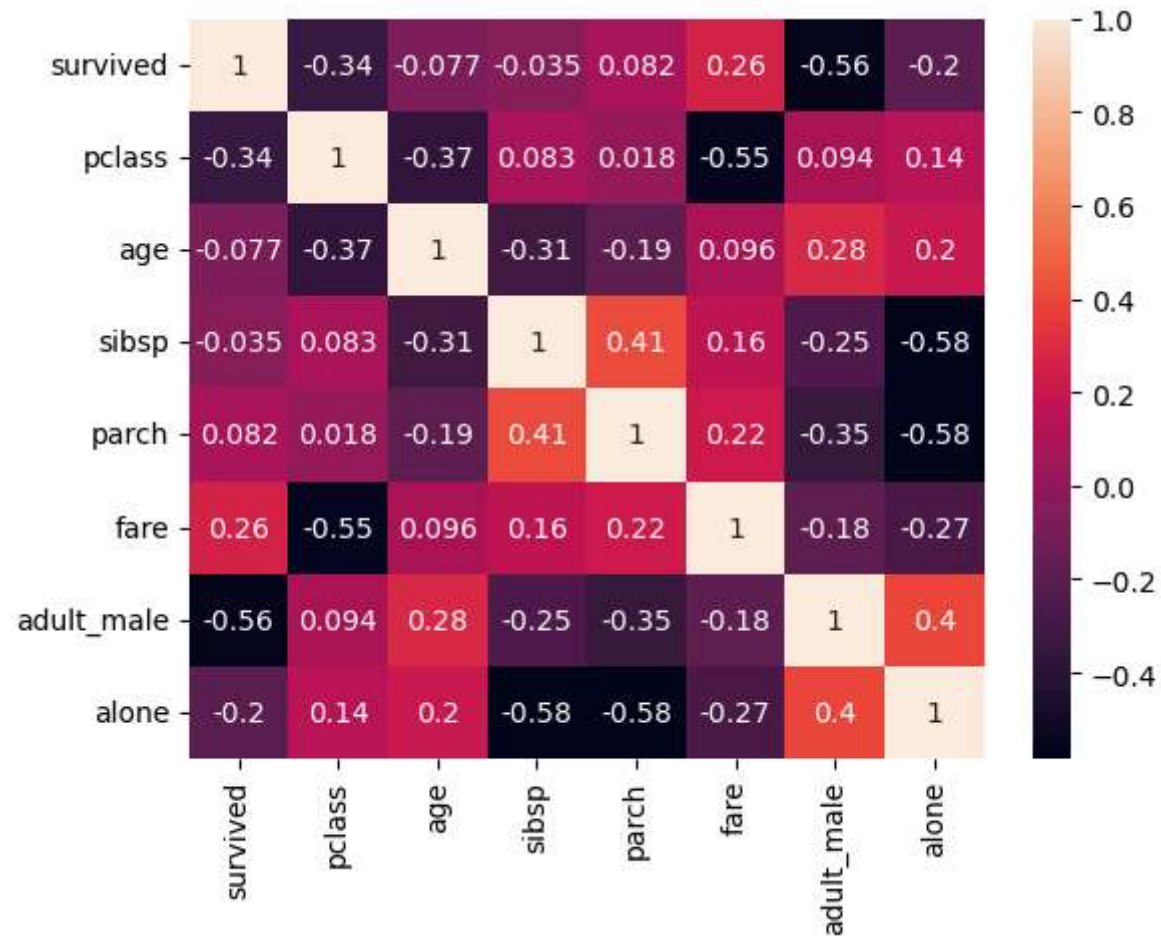
```
In [18]: data=sns.load_dataset("titanic")
corr = data.corr()
corr
```

Out[18]:

	survived	pclass	age	sibsp	parch	fare	adult_male	alone
survived	1.000000	-0.338481	-0.077221	-0.035322	0.081629	0.257307	-0.557080	-0.203367
pclass	-0.338481	1.000000	-0.369226	0.083081	0.018443	-0.549500	0.094035	0.135207
age	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	0.096067	0.280328	0.198270
sibsp	-0.035322	0.083081	-0.308247	1.000000	0.414838	0.159651	-0.253586	-0.584471
parch	0.081629	0.018443	-0.189119	0.414838	1.000000	0.216225	-0.349943	-0.583398
fare	0.257307	-0.549500	0.096067	0.159651	0.216225	1.000000	-0.182024	-0.271832
adult_male	-0.557080	0.094035	0.280328	-0.253586	-0.349943	-0.182024	1.000000	0.404744
alone	-0.203367	0.135207	0.198270	-0.584471	-0.583398	-0.271832	0.404744	1.000000

```
In [19]: sns.heatmap(corr, annot=True) # this will give you a basic heat map
```

```
Out[19]: <AxesSubplot:>
```



```
In [ ]:
```